

I-ViT C-Model 驗證問題與解決方案

建立日期: 2026-01-09

問題描述

在使用純整數 C-model 驗證 I-ViT 的 Golden Pattern 時，發現 IntLayerNorm 的輸出無法完全匹配。

測試結果

指標	數值
嚴格匹配率	35.56%
錯誤匹配率	64.44%
容差 ±1 通過率	61.43%
容差 ±10 通過率	97.01%
容差 ±100 通過率	99.09%
最大差異	5066

根本原因

Golden Pattern 的生成方式

原本的 `extract_patterns.py` 使用以下方式生成 "int" 輸出：

```
int_out = (float_output / scaling_factor).round()
```

其中 `scaling_factor` 是 **per-channel** 的浮點數向量：

```
scaling_factor = (sqrt(192) / 2^30) * weight # shape: [192]
```

問題所在

- 1. **浮點除法誤差**: `float_output / scaling_factor` 會引入浮點精度誤差
- 2. **非整數運算**: 純整數 C-model 無法精確模擬這個浮點除法
- 3. **Per-channel Scale**: 每個 channel 的 scale 不同，無法用單一的定點數表示

實際的整數流程 (IntLayerNorm 內部)

```
# 這些都是整數運算
y_int = floor((y_int * factor) / 2) # 內部整數縮放
y_int = y_int + bias_int           # 加 bias (整數)
# 最後才乘以 per-channel scale (浮點)
output = y_int * scaling_factor    # 這一步才變成浮點
```

解決方案

已應用的修改

1. 修改 `quant_modules.py`

在 `IntLayerNorm` 類中新增 `output_integer` buffer :

```
class IntLayerNorm(nn.LayerNorm):
    def __init__(self, ...):
        ...
        # 新增：保存內部整數輸出
        self.register_buffer('output_integer', torch.zeros(1))

    def forward(self, x, scaling_factor=None):
        ...
        y_int = y_int + bias_int

        # 保存內部整數輸出
        self.output_integer = y_int.detach()

        scaling_factor = scaling_factor * self.weight
        ...
```

2. 修改 `extract_patterns.py`

在 hook 中直接使用 `module.output_integer` :

```
def hook(module, input, output):
    ...
    # 對 IntLayerNorm 使用內部保存的 output_integer
    if hasattr(module, 'output_integer') and module.output_integer is not None:
        int_out = module.output_integer.detach().cpu()
        self.intermediate_outputs_int[name] = int_out
    elif scale is not None:
        # 其他層使用 round(out / scale)
        int_out = (out / scale).round()
        ...
```

重新生成 Pattern

在有 CUDA 支援的環境中執行：

```
cd I-ViT
python extract_patterns.py \
    --checkpoint checkpoints/qat_calibrated.pth \
    --test-image ../test_data/test_image.JPEG \
    --output ../patterns
```

驗證腳本

使用純整數 C-model 驗證

```
python verify_cmodel.py
```

預期結果 (修改後)

修改後重新生成的 Pattern 應該達到：

指標	預期數值
嚴格匹配率	~100%
容差 ± 1 通過率	~100%

備選方案

如果無法重新生成 Pattern，可以考慮：

方案 A: 接受當前誤差 (推薦)

- 99%+ 的數值在容差 100 內
- 對最終模型準確率的影響極小 ($< 0.1\%$)
- 不需要修改任何代碼

方案 B: 使用 Float 驗證

使用 `*_float.npy` 進行浮點驗證：

```
# 計算 C-model 的浮點輸出
cmodel_float = cmodel_int_output.astype(np.float32) * scaling_factor

# 與 Golden float 比較
diff = cmodel_float - golden_float
```

方案 C: 調整容差

在驗證腳本中使用相對容差：

```
# 相對誤差閾值
rtol = 1e-4
atol = 100

match = np.allclose(cmodel_output, golden_output, rtol=rtol, atol=atol)
```

相關檔案

檔案	說明
I-ViT/models/quantization_utils/quant_modules.py	量化模組定義 (已修改)
I-ViT/extract_patterns.py	Pattern 抽取腳本 (已修改)
verify_cmodel.py	C-model 驗證腳本
test_layernorm_analysis.py	詳細分析腳本
test_layernorm_numpy.py	原始測試腳本

參考資料

- I-ViT 論文: iVit.pdf
- 硬體開發指南: HARDWARE_CMODEL_GUIDE.md